

Chapter 10 interpolation and extrapolation

April 9, 2024

0.1 Chapter 10 Making predications by interpolation and extrapolation

Interpolation refers to the process of generating data points between already existing data points. Extrapolation is the process of generating points outside a given set of known data points. (inter and extra are derived from Latin words meaning ‘between’ and ‘outside’ respectively) [click here](#) It is rarely to find interpolate/extrapolate chapter in a book for beginners or intermediate in Python course. However, the likelihood of confronting interpolation problem in your computing life should be above the 99% and that’s why I feel that we must go through this together so you will remember me later .

As always there are several ways and modules for a such task in Python. In that chapter, we will discuss through examples three ways of doing interpolation.

0.2 `scipy.interpolate.interp1d`

class `scipy.interpolate.interp1d(x, y, kind='linear', axis=- 1, copy=True, bounds_error=None, fill_value=nan, assume_sorted=False)` [\[source\]](#)

x and y are arrays of values used to approximate some function $f: y = f(x)$. This class returns a function whose call method uses interpolation to find the value of new points.

The most important input values in this function are the following: * x(N,) array_like: A 1-D array of real values. * y(...,N,...) array_like: A N-D array of real values. The length of y along the interpolation axis must be equal to the length of x. * kind: str or int, optional Specifies the kind of interpolation as a string (‘linear’, ‘nearest’, ‘zero’, ‘slinear’, ‘quadratic’, ‘cubic’, ‘previous’, ‘next’, where ‘zero’, ‘slinear’, ‘quadratic’ and ‘cubic’ refer to a spline interpolation of zeroth, first, second or third order; ‘previous’ and ‘next’ simply return the previous or next value of the point) or as an integer specifying the order of the spline interpolator to use. Default is ‘linear’. * fill_value: array-like or (array-like, array_like) or “extrapolate”, optional: this value will be used to fill in for requested points outside of the data range. If not provided, then the default is NaN.

```
[1]: import matplotlib.pyplot as plt
from scipy import interpolate
import numpy as np

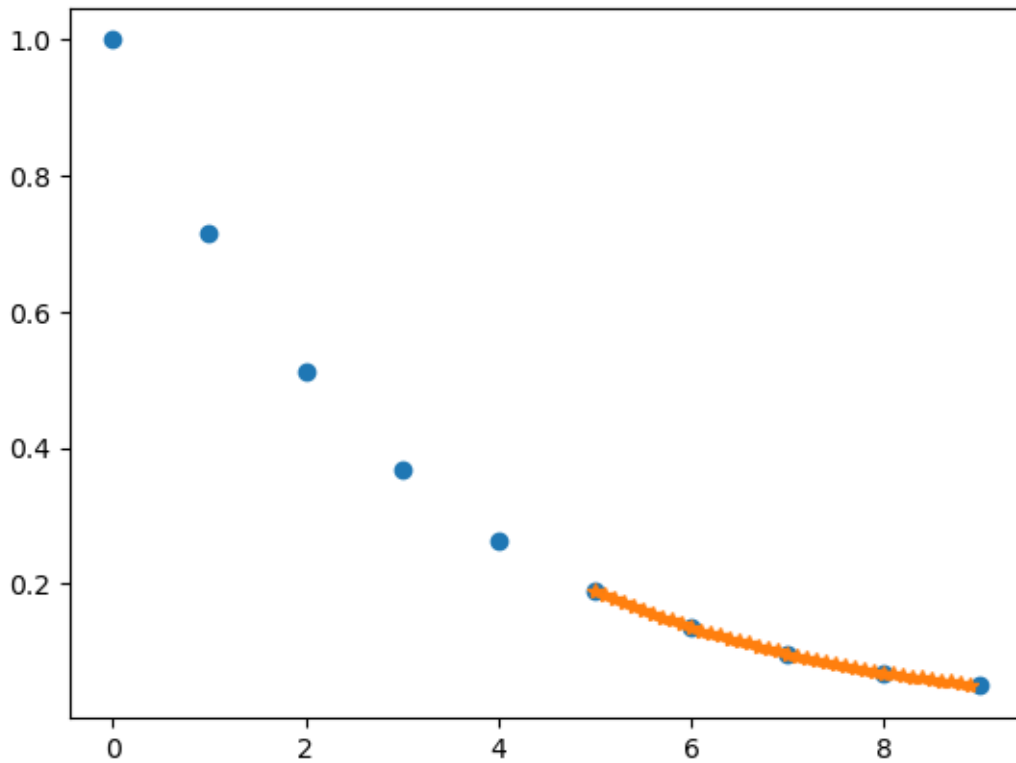
x = np.arange(0, 10)
y = np.exp(-x/3.0)
f = interpolate.interp1d(x, y)
```

```

xnew = np.arange(5, 9, 0.1)
ynew = f(xnew)    # use interpolation function returned by `interp1d`

plt.plot(x, y, 'o', xnew, ynew, '-*')
plt.show()

```



```

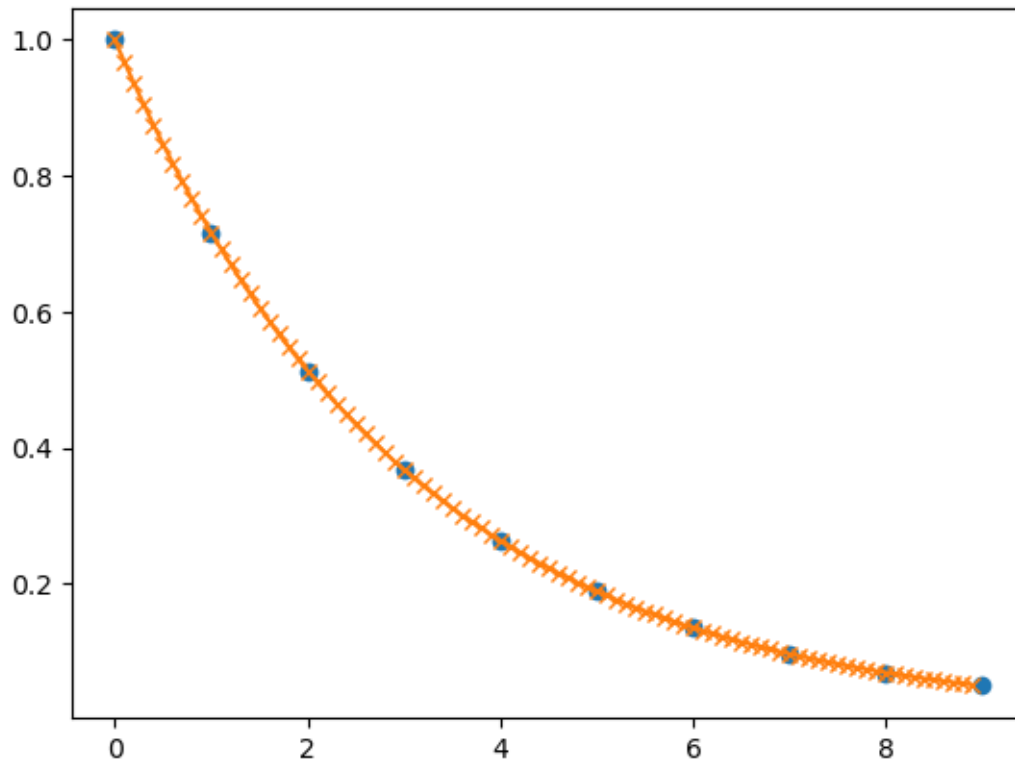
[2]: import matplotlib.pyplot as plt
from scipy import interpolate
import numpy as np

x = np.arange(0, 10)
y = np.exp(-x/3.0)
f = interpolate.interp1d(x, y, kind = 'cubic')

xnew = np.arange(0, 9, 0.1)
ynew = f(xnew)

plt.plot(x, y, 'o', xnew, ynew, '-x')
plt.show()

```



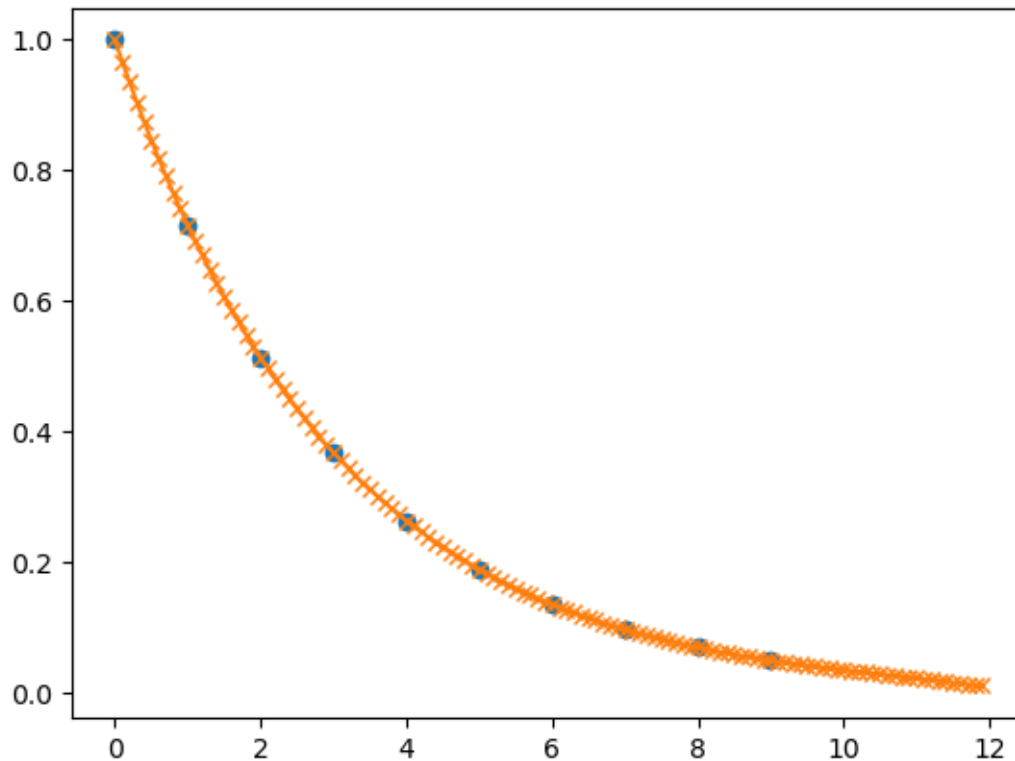
We can use the attribute `fill_value= 'extrapolate'` for extrapolation.

```
[3]: import matplotlib.pyplot as plt
from scipy import interpolate
import numpy as np

x = np.arange(0, 10)
y = np.exp(-x/3.0)
f = interpolate.interp1d(x, y, kind = 'cubic', fill_value= 'extrapolate')

xnew = np.arange(0, 12, 0.1)
ynew = f(xnew)

plt.plot(x, y, 'o', xnew, ynew, '-x')
plt.show()
```



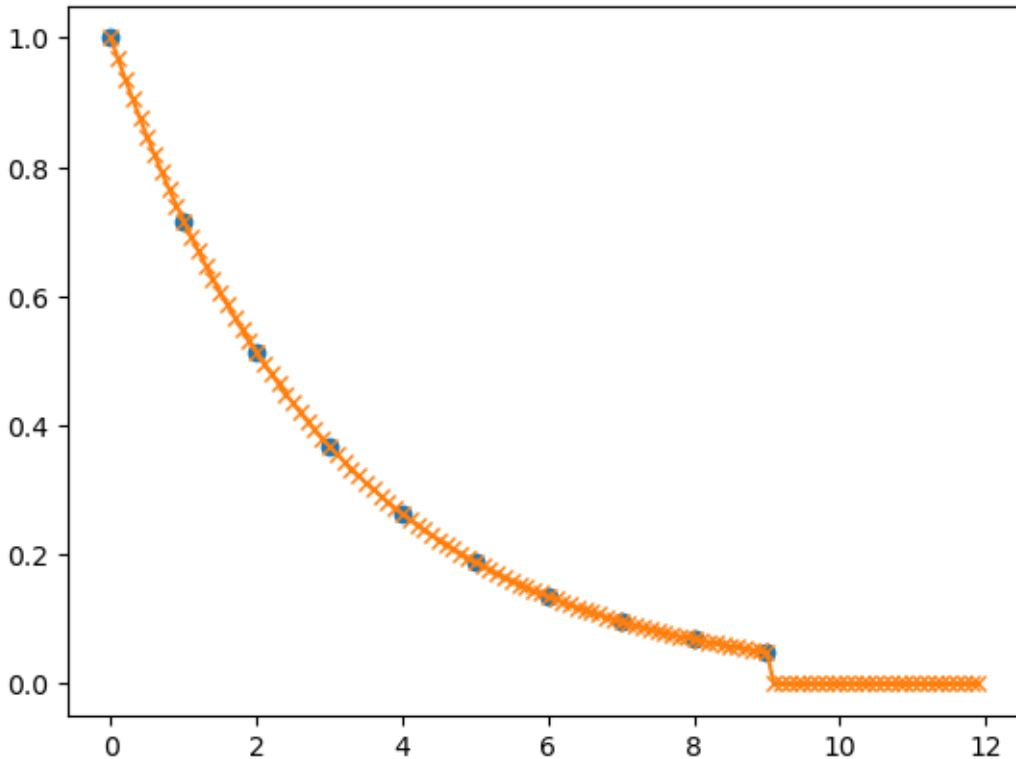
If a two-element tuple, then the first element is used as a fill value for $x_{\text{new}} < x[0]$ and the second element is used for $x_{\text{new}} > x[-1]$. In that case, remember to disable the bounds error using the attribute `bounds_error=False`:

```
[4]: import matplotlib.pyplot as plt
from scipy import interpolate
import numpy as np

x = np.arange(0, 10)
y = np.exp(-x/3.0)
f = interpolate.interp1d(x, y, kind = 'cubic', fill_value= (0,0)\
                        ,bounds_error=False)

xnew = np.arange(0, 12, 0.1)
ynew = f(xnew)

plt.plot(x, y, 'o', xnew, ynew, '-x')
plt.show()
```



0.3 numpy.interp:

`numpy.interp(x, xp, fp, left=None, right=None, period=None)[source]`

One-dimensional linear interpolation. Returns the one-dimensional piecewise linear interpolant to a function with given discrete data points (xp, fp), evaluated at x.

Parameters:

- **x:** array_like. The x-coordinates at which to evaluate the interpolated values.
- **xp:** 1-D sequence of floats. The x-coordinates of the data points, must be increasing if argument period is not specified. Otherwise, xp is internally sorted after normalizing the periodic boundaries with $xp = xp \% \text{period}$.
- **fp:** 1-D sequence of float or complex. The y-coordinates of the data points, same length as xp.
- **left:** optional float or complex corresponding to fp: Value to return for $x < xp[0]$, default is `fp[0]`.
- **right:** optional float or complex corresponding to fp Value to return for $x > xp[-1]$, default is `fp[-1]`.

Returns: y: float or complex (corresponding to fp) or ndarray. The interpolated values, same shape as x.

```
[5]: xp = np.arange(0, 10)
      fp = np.exp(-x/3.0)
      np.interp(11, xp, fp)
```

```
[5]: 0.049787068367863944
```

Extrapolation with specific values:

```
[6]: xp = np.arange(0, 10)
      fp = np.exp(-x/3.0)
      print(np.interp(-1, xp, fp, left = 0, right = 1))
      print(np.interp(11, xp, fp, left = 0, right = 1))
```

```
0.0
```

```
1.0
```

Another example of interpolation to the sine function:

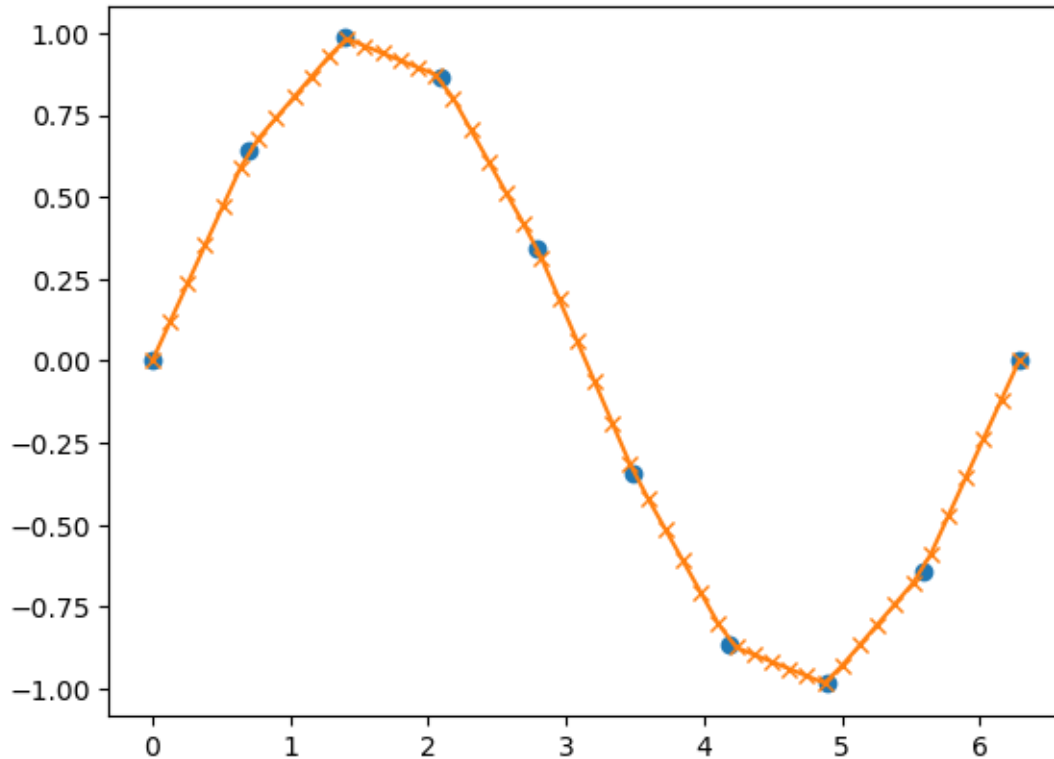
```
[7]: x = np.linspace(0, 2*np.pi, 10)
      y = np.sin(x)

      xvals = np.linspace(0, 2*np.pi, 50)
      yinterp = np.interp(xvals, x, y)

      import matplotlib.pyplot as plt

      plt.plot(x, y, 'o')
      plt.plot(xvals, yinterp, '-x')

      plt.show()
```



I don't recommend this method because it is limited to the linear interpolation method.

0.4 `pandas.DataFrame.interpolate`

`DataFrame.interpolate(method='linear', axis=0, limit=None, inplace=False, limit_direction=None, limit_area=None, downcast=None, kwargs)` *Parameters:* `method`: str, default 'linear': But we can use others like 'nearest', 'zero', 'slinear', 'quadratic', 'cubic', 'spline', 'barycentric', 'polynomial': Passed to `scipy.interpolate.interp1d`. These methods use the numerical values of the index. Both 'polynomial' and 'spline' require that you also specify an order (int), e.g. `df.interpolate(method='polynomial', order=5)`.

```
[8]: import pandas as pd
import numpy as np
s = pd.Series([0, 1, np.nan, 3])
print(s)
s.interpolate()
```

```
0    0.0
1    1.0
2    NaN
3    3.0
dtype: float64
```

```
[8]: 0    0.0
      1    1.0
      2    2.0
      3    3.0
      dtype: float64
```

```
[9]: import pandas as pd
      s = pd.Series([0, 1, np.nan, 3])
      print(s)
      s.interpolate(method='polynomial', order = 2)
```

```
0    0.0
1    1.0
2    NaN
3    3.0
dtype: float64
```

```
[9]: 0    0.0
      1    1.0
      2    2.0
      3    3.0
      dtype: float64
```

```
[10]: import numpy as np
       import pandas as pd

       exam_data = {'name': ['Anastasia', 'Dima', 'Katherine', 'James', 'Emily',
                               ↪ 'Michael', 'Matthew', 'Laura', 'Kevin', 'Jonas'],

                     'score': [12.5, 9, 16.5, np.nan, 9, 20, 14.5, np.nan, 8, 19],

                     'attempts': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],

                     'qualify': ['yes', 'no', 'yes', 'no', 'no', 'yes', 'yes', 'no', 'no',
                               ↪ 'yes']}

       labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']

       df = pd.DataFrame(exam_data , index=labels)
       print(df)

       df = df.interpolate(method='linear')
       print(df)
```

	name	score	attempts	qualify
a	Anastasia	12.5	1	yes
b	Dima	9.0	3	no

c	Katherine	16.5	2	yes
d	James	NaN	3	no
e	Emily	9.0	2	no
f	Michael	20.0	3	yes
g	Matthew	14.5	1	yes
h	Laura	NaN	1	no
i	Kevin	8.0	2	no
j	Jonas	19.0	1	yes
	name	score	attempts	qualify
a	Anastasia	12.50	1	yes
b	Dima	9.00	3	no
c	Katherine	16.50	2	yes
d	James	12.75	3	no
e	Emily	9.00	2	no
f	Michael	20.00	3	yes
g	Matthew	14.50	1	yes
h	Laura	11.25	1	no
i	Kevin	8.00	2	no
j	Jonas	19.00	1	yes

[]: